SOFTWARE:

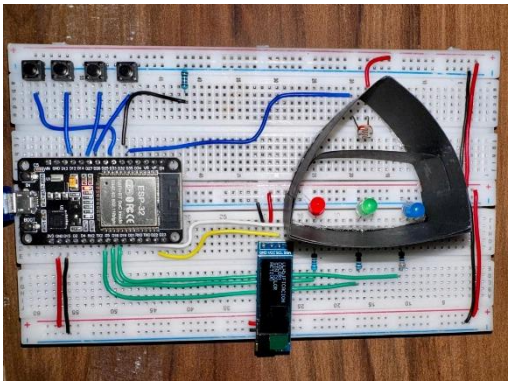
```
#include <stdio.h>
#include <string.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"
#include "driver/adc.h"
#include "driver/i2c.h"

#define LED_R GPIO_NUM_5
#define LED_G GPIO_NUM_17
#define LED_B GPIO_NUM_18

#define BTN_1 GPIO_NUM_25
#define BTN_2 GPIO_NUM_13
#define BTN_3 GPIO_NUM_14

#define ADC_CHANNEL ADC1_CHANNEL_4

#define I2C_MASTER_SCL_IO 22
#define I2C_MASTER_SDA_IO 21
#define I2C_MASTER_NUM I2C_NUM_0
#define I2C_MASTER_FREQ_HZ 100000
#define OLED_ADDR 0x3C

int base, R, G, B;
int Rf, Gf, Bf;

int modo = 0;
int congelado = 0;
int escala = 0;

float Rp, Gp, Bp;

int estado_anterior_b1 = 1;
int estado_anterior_b2 = 1;
int estado_anterior_b3 = 1;

void i2c_master_init(void)
{
   i2c_config_t conf = {
      .mode = I2C_MODE_MASTER,
      .sda_io_num = I2C_MASTER_SDA_IO,
      .scl_io_num = I2C_MASTER_SCL_IO,
      .sda_pullup_en = GPIO_PULLUP_ENABLE,
      .scl_pullup_en = GPIO_PULLUP_ENABLE,
      .master.clk_speed = I2C_MASTER_FREQ_HZ
   };

   i2c_param_config(I2C_MASTER_NUM, &conf);
   i2c_driver_install(I2C_MASTER_NUM, conf.mode, 0,
0, 0);
}

void oled_command(uint8_t cmd)
{
   uint8_t data[2] = {0x00, cmd};
   i2c_master_write_to_device(I2C_MASTER_NUM,
OLED_ADDR, data, 2, pdMS_TO_TICKS(1000));
}

void oled_data(uint8_t *data, size_t len)
{
   uint8_t buffer[129];
   buffer[0] = 0x40;
   memcpy(&buffer[1], data, len);
   i2c_master_write_to_device(I2C_MASTER_NUM,
OLED_ADDR, buffer, len + 1, pdMS_TO_TICKS(1000));
}

void oled_init(void)
{
   vTaskDelay(pdMS_TO_TICKS(100));

   oled_command(0xAE);
   oled_command(0xD5);
   oled_command(0x80);
   oled_command(0xA8);
   oled_command(0x1F);
   oled_command(0xD3);
   oled_command(0x00);
   oled_command(0x40);
   oled_command(0x8D);
   oled_command(0x14);
   oled_command(0x20);
   oled_command(0x00);
   oled_command(0xA1);
   oled_command(0xC8);
   oled_command(0xDA);
   oled_command(0x02);
   oled_command(0x81);
   oled_command(0xCF);
   oled_command(0xD9);
   oled_command(0xF1);
   oled_command(0xDB);
   oled_command(0x40);
   oled_command(0xA4);
   oled_command(0xA6);
   oled_command(0xAF);
}

void oled_clear(void)
{
   for (int page = 0; page < 4; page++)
   {
      oled_command(0xB0 + page);
      oled_command(0x00);
      oled_command(0x10);

      uint8_t data[128];
      memset(data, 0x00, 128);
      oled_data(data, 128);
   }
}

void oled_set_cursor(uint8_t page, uint8_t col)
{
   oled_command(0xB0 + page);
   oled_command(0x00 + (col & 0x0F));
   oled_command(0x10 + ((col >> 4) & 0x0F));
}

void get_char(char c, uint8_t font[5])
{
   memset(font, 0x00, 5);
```

```
    switch (c)
    {
        case '0': {uint8_t f[5]={0x3E,0x51,0x49,0x45,0x3E};
memcpy(font,f,5);} break;
        case '1': {uint8_t f[5]={0x00,0x42,0x7F,0x40,0x00};
memcpy(font,f,5);} break;
        case '2': {uint8_t f[5]={0x42,0x61,0x51,0x49,0x46};
memcpy(font,f,5);} break;
        case '3': {uint8_t f[5]={0x21,0x41,0x45,0x4B,0x31};
memcpy(font,f,5);} break;
        case '4': {uint8_t f[5]={0x18,0x14,0x12,0x7F,0x10};
memcpy(font,f,5);} break;
        case '5': {uint8_t f[5]={0x27,0x45,0x45,0x45,0x39};
memcpy(font,f,5);} break;
        case '6': {uint8_t f[5]={0x3C,0x4A,0x49,0x49,0x30};
memcpy(font,f,5);} break;
        case '7': {uint8_t f[5]={0x01,0x71,0x09,0x05,0x03};
memcpy(font,f,5);} break;
        case '8': {uint8_t f[5]={0x36,0x49,0x49,0x49,0x36};
memcpy(font,f,5);} break;
        case '9': {uint8_t f[5]={0x06,0x49,0x49,0x29,0x1E};
memcpy(font,f,5);} break;


        case 'A': {uint8_t f[5]={0x7E,0x11,0x11,0x11,0x7E};
memcpy(font,f,5);} break;
        case 'B': {uint8_t f[5]={0x7F,0x49,0x49,0x49,0x36};
memcpy(font,f,5);} break;
        case 'C': {uint8_t f[5]={0x3E,0x41,0x41,0x41,0x22};
memcpy(font,f,5);} break;
        case 'D': {uint8_t f[5]={0x7F,0x41,0x41,0x22,0x1C};
memcpy(font,f,5);} break;
        case 'E': {uint8_t f[5]={0x7F,0x49,0x49,0x49,0x41};
memcpy(font,f,5);} break;
        case 'F': {uint8_t f[5]={0x7F,0x09,0x09,0x09,0x01};
memcpy(font,f,5);} break;
        case 'G': {uint8_t f[5]={0x3E,0x41,0x49,0x49,0x7A};
memcpy(font,f,5);} break;
        case 'I': {uint8_t f[5]={0x00,0x41,0x7F,0x41,0x00};
memcpy(font,f,5);} break;
        case 'J': {uint8_t f[5]={0x20,0x40,0x41,0x3F,0x01};
memcpy(font,f,5);} break;
        case 'L': {uint8_t f[5]={0x7F,0x40,0x40,0x40,0x40};
memcpy(font,f,5);} break;
        case 'M': {uint8_t f[5]={0x7F,0x02,0x0C,0x02,0x7F};
memcpy(font,f,5);} break;
        case 'N': {uint8_t f[5]={0x7F,0x04,0x08,0x10,0x7F};
memcpy(font,f,5);} break;
        case 'O': {uint8_t f[5]={0x3E,0x41,0x41,0x41,0x3E};
memcpy(font,f,5);} break;
        case 'P': {uint8_t f[5]={0x7F,0x09,0x09,0x09,0x06};
memcpy(font,f,5);} break;
        case 'R': {uint8_t f[5]={0x7F,0x09,0x19,0x29,0x46};
memcpy(font,f,5);} break;
        case 'S': {uint8_t f[5]={0x46,0x49,0x49,0x49,0x31};
memcpy(font,f,5);} break;
        case 'T': {uint8_t f[5]={0x01,0x01,0x7F,0x01,0x01};
memcpy(font,f,5);} break;
        case 'U': {uint8_t f[5]={0x3F,0x40,0x40,0x40,0x3F};
memcpy(font,f,5);} break;
        case 'V': {uint8_t f[5]={0x1F,0x20,0x40,0x20,0x1F};
memcpy(font,f,5);} break;
        case 'W': {uint8_t f[5]={0x7F,0x20,0x18,0x20,0x7F};
memcpy(font,f,5);} break;
        case 'Z': {uint8_t f[5]={0x61,0x51,0x49,0x45,0x43};
memcpy(font,f,5);} break;


        case ':': {uint8_t f[5]={0x00,0x36,0x36,0x00,0x00};
memcpy(font,f,5);} break;
        case '-': {uint8_t f[5]={0x08,0x08,0x08,0x08,0x08};
memcpy(font,f,5);} break;
        case '.': {uint8_t f[5]={0x00,0x60,0x60,0x00,0x00};
memcpy(font,f,5);} break;
        case '%': {uint8_t f[5]={0x23,0x13,0x08,0x64,0x62};
memcpy(font,f,5);} break;
        case ' ': default: break;
    }
}


void oled_write_char(char c)
{
    uint8_t font[5];
    get_char(c, font);


    uint8_t data[6] = {font[0], font[1], font[2], font[3], font[4],
0x00};
    oled_data(data, 6);
}


void oled_write_string(const char *str)
{
    while (*str)
    {
        oled_write_char(*str++);
    }
}


void oled_print_line(uint8_t line, const char *text)
{
    oled_set_cursor(line, 0);


    uint8_t clear[128];
    memset(clear, 0x00, 128);
    oled_data(clear, 128);


    oled_set_cursor(line, 0);
    oled_write_string(text);
}


const char* detectar_color(void)
{
    if (Rf < 80 && Gf < 80 && Bf < 80)
    {
        return "SIN COLOR";
    }


    if (Gf >= Rf && Gf >= Bf)
    {
        return "VERDE";
    }
    else if (Rf >= Gf && Rf >= Bf)
    {
        return "ROJO";
    }
    else
    {
        return "AZUL";
    }
}


void leer_botones(void)
{
    int b1 = gpio_get_level(BTN_1);
    int b2 = gpio_get_level(BTN_2);
    int b3 = gpio_get_level(BTN_3);
```

```
    if (b1 == 0 && estado_anterior_b1 == 1)
    {
        modo = !modo;
        printf("CAMBIO DE MODO\n");
    }

    estado_anterior_b1 = b1;

    if (b2 == 0 && estado_anterior_b2 == 1)
    {
        congelado = !congelado;
        printf(congelado ? "CONGELADO\n" :
"REANUDADO\n");
    }

    estado_anterior_b2 = b2;

    if (b3 == 0 && estado_anterior_b3 == 1)
    {
        escala = !escala;
        printf(escala ? "ESCALA: %%\n" : "ESCALA:
RAW\n");
    }

    estado_anterior_b3 = b3;
}

void medir_color(void)
{
    gpio_set_level(LED_R, 0);
    gpio_set_level(LED_G, 0);
    gpio_set_level(LED_B, 0);
    vTaskDelay(pdMS_TO_TICKS(200));
    base = adc1_get_raw(ADC_CHANNEL);

    gpio_set_level(LED_R, 1);
    vTaskDelay(pdMS_TO_TICKS(200));
    R = adc1_get_raw(ADC_CHANNEL);
    gpio_set_level(LED_R, 0);

    gpio_set_level(LED_G, 1);
    vTaskDelay(pdMS_TO_TICKS(200));
    G = adc1_get_raw(ADC_CHANNEL);
    gpio_set_level(LED_G, 0);

    gpio_set_level(LED_B, 1);
    vTaskDelay(pdMS_TO_TICKS(200));
    B = adc1_get_raw(ADC_CHANNEL);
    gpio_set_level(LED_B, 0);

    Rf = R - base;
    Gf = G - base;
    Bf = B - base;

    if (Rf < 0) Rf = 0;
    if (Gf < 0) Gf = 0;
    if (Bf < 0) Bf = 0;

//AJUSTES DE SENSIBILIDAD DE LOS CANALES EN
FUNCION
//DE LAS RESISTENCIAS Y LOS RESULTADOS

//Ajuste de la sensibilidad del verde
Gf = Gf * 1;

//Ajuste de la sensibilidad del azul
Bf = Bf * 1;

//Ajuste de la sensibilidad del rojo
Bf = Bf * 0.70;



}

void calcular_porcentajes(void)
{
    int suma = Rf + Gf + Bf;
    if (suma == 0) suma = 1;

    Rp = (Rf * 100.0f) / suma;
    Gp = (Gf * 100.0f) / suma;
    Bp = (Bf * 100.0f) / suma;
}

void app_main(void)
{
    gpio_reset_pin(LED_R);
    gpio_set_direction(LED_R, GPIO_MODE_OUTPUT);

    gpio_reset_pin(LED_G);
    gpio_set_direction(LED_G, GPIO_MODE_OUTPUT);

    gpio_reset_pin(LED_B);
    gpio_set_direction(LED_B, GPIO_MODE_OUTPUT);

    gpio_reset_pin(BTN_1);
    gpio_set_direction(BTN_1, GPIO_MODE_INPUT);
    gpio_set_pull_mode(BTN_1, GPIO_PULLUP_ONLY);

    gpio_reset_pin(BTN_2);
    gpio_set_direction(BTN_2, GPIO_MODE_INPUT);
    gpio_set_pull_mode(BTN_2, GPIO_PULLUP_ONLY);

    gpio_reset_pin(BTN_3);
    gpio_set_direction(BTN_3, GPIO_MODE_INPUT);
    gpio_set_pull_mode(BTN_3, GPIO_PULLUP_ONLY);

    adc1_config_width(ADC_WIDTH_BIT_12);
    adc1_config_channel_atten(ADC_CHANNEL,
ADC_ATTEN_DB_11);

    i2c_master_init();
    oled_init();
    oled_clear();

    char linea1[25];
    char linea2[25];
    char linea3[25];
    char linea4[25];

    while (1)
    {
        leer_botones();
```

```
        if (!congelado)
        {
            medir_color();
        }

        calcular_porcentajes();

        const char *resultado = detectar_color();

        if (modo == 0)
        {
            if (escala == 0)
            {
                printf("MODO: MEDICION RAW\n");
                printf("R: %d | V: %d | A: %d\n", Rf, Gf, Bf);

                sprintf(linea1, "MEDICION RAW");
                sprintf(linea2, "R:%d", Rf);
                sprintf(linea3, "V:%d", Gf);
                sprintf(linea4, "A:%d", Bf);
            }
            else
            {
                printf("MODO: MEDICION %%\n");
                printf("R: %.1f%% | V: %.1f%% | A:
%.1f%%\n", Rp, Gp, Bp);

                sprintf(linea1, "MEDICION %%");
                sprintf(linea2, "R:%.1f%%", Rp);
                sprintf(linea3, "V:%.1f%%", Gp);
                sprintf(linea4, "A:%.1f%%", Bp);
            }
        }
        else
        {
            printf("MODO: CLASIFICACION\n");
            printf("COLOR: %s\n", resultado);

            sprintf(linea1, "CLASIFICACION");
            sprintf(linea2, "COLOR:");
            sprintf(linea3, "%s", resultado);

            if (congelado)
                sprintf(linea4, "PAUSA");
            else
                sprintf(linea4, "ACTIVO");
        }

        printf("-----------------------------\n");

        oled_print_line(0, linea1);
        oled_print_line(1, linea2);
        oled_print_line(2, linea3);
        oled_print_line(3, linea4);

        vTaskDelay(pdMS_TO_TICKS(500));
    }
}
```

HARDWARE:



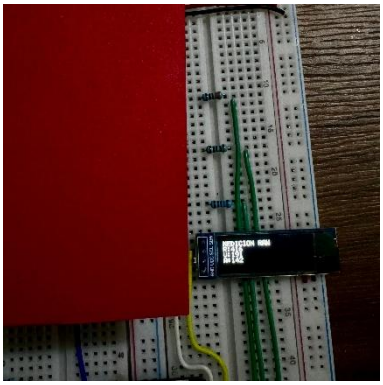
Montaje del circuito en protoboard

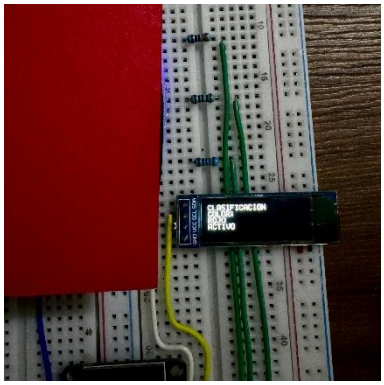
Objetos utilizados para pruebas

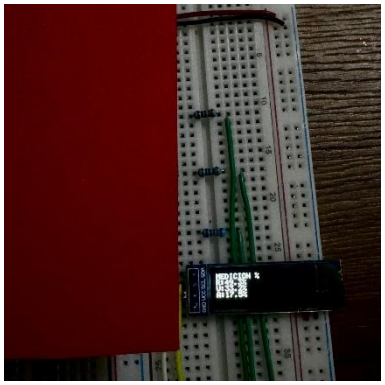
PRUEBAS DE FUNCIONAMIENTO

PRUEBA 1: OBJETOS PLANOS

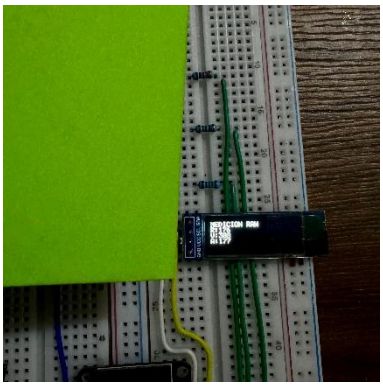
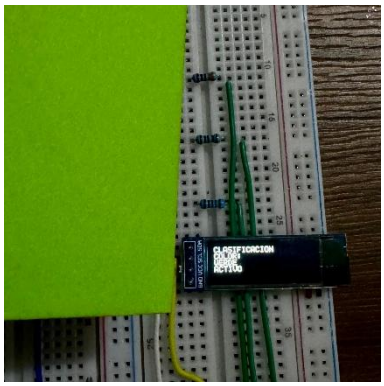
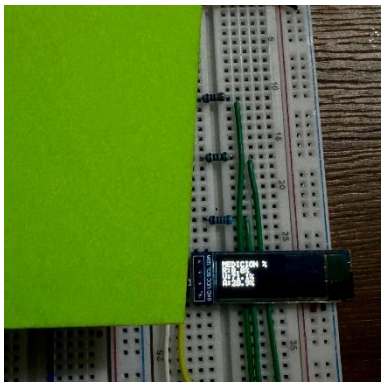
Objeto Rojo: Medición Raw, Clasificación y Medición %



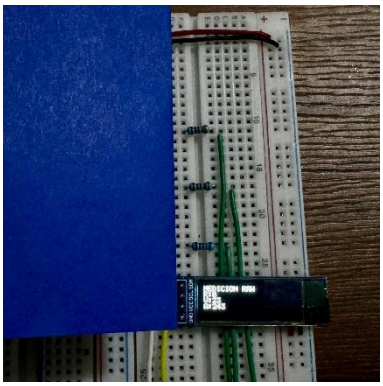
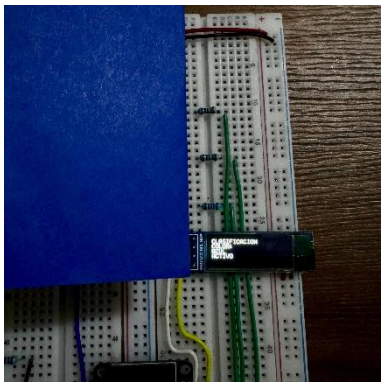
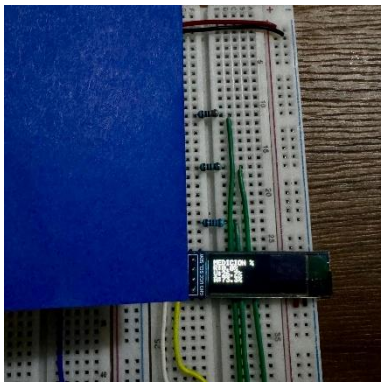




Objeto Verde: Medición Raw, Clasificación y Medición %

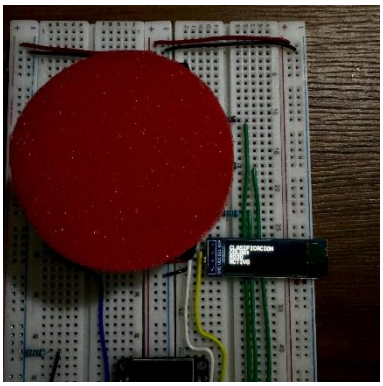
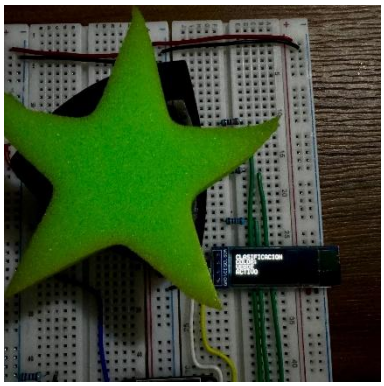
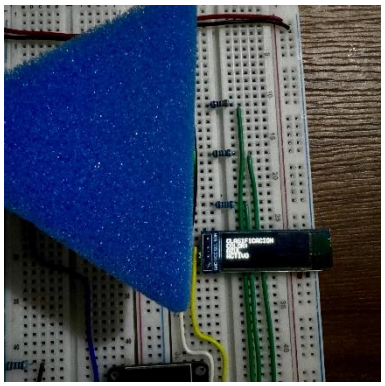
Objeto Azul: Medición Raw, Clasificación y Medición %

COLOR DEL OBJETO	RAW	CLASIFICACIÓN	%
Rojo	R:416 V:191 A:142	Rojo	R:49.3% V:33.2% A:17.5%
Verde	R:176 V:308 A:177	Verde	R:0.0% V:71.1% A:28.9%
Azul	R:0 V:23 A:243	Azul	R:0.0% V:26.7% A:73.3%

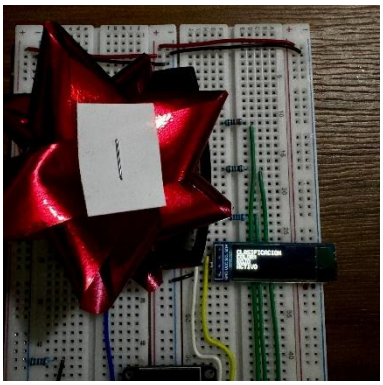
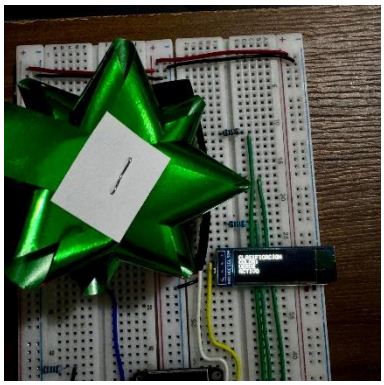
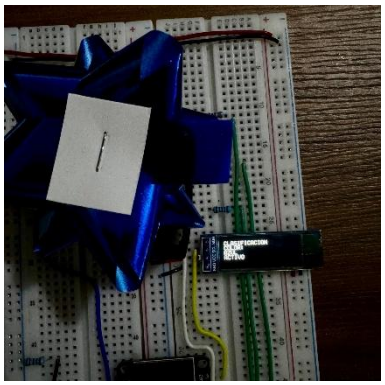
PRUEBA 2: OBJETOS POROSOS

Clasificación: Rojo, Verde y Azul

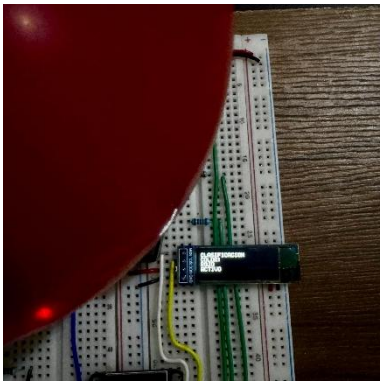
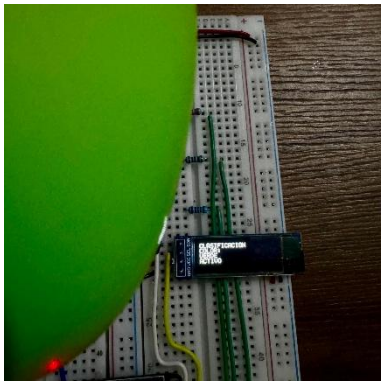
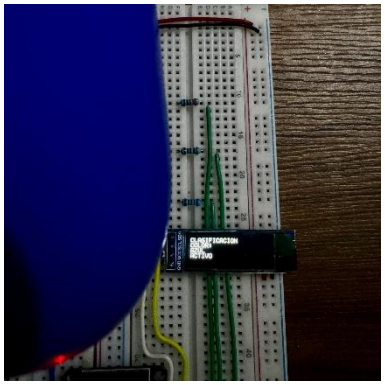
PRUEBA 3: OBJETOS IRREGULARES

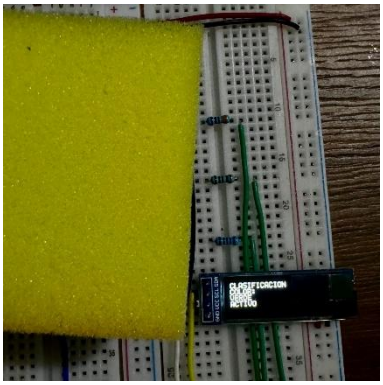
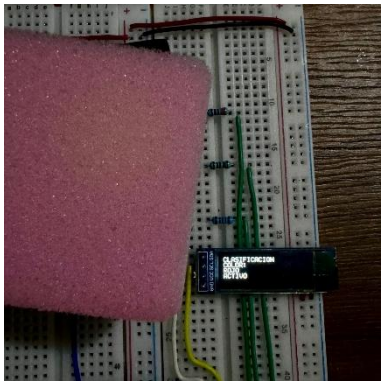
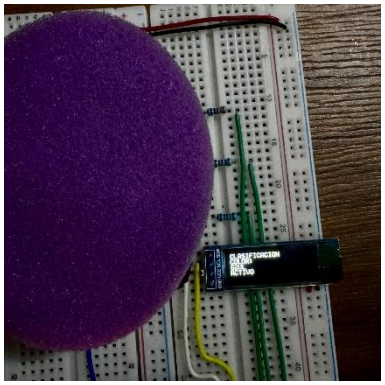
Clasificación: Rojo, Verde y Azul

PRUEBA 4: OBJETOS ESFÉRICOS

Clasificación: Rojo, Verde y Azul

PRUEBA 4: OBJETOS ESFÉRICOS

Clasificación de amarillo, rosa y morado, dando como resultados rojo, verde y azul, respectivamente.